

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «АППАРАТНО-ПРОГРАММНОЕ
ОБЕСПЕЧЕНИЕ ЭВМ И СЕТЕЙ»

Тема: «Изучение аппаратно-программной
архитектуры процессоров семейства Intel – макроопределения»

Выполнил:
Студент 4-го курса
Группы АС-66
Ануфриенко М. А.
Проверила:
Кулик А. Д.

Цель: изучение приемов разработки макроопределений использования их в программах.

1. Изучить состав и средства задания макроопределений.
2. Написать макроопределение, реализующее функцию заданного преподавателем варианта работы.
3. Выполнить проверку работоспособности разработанного макроопределения.

Вариант 1

1. Подсчитать количество вхождений заданного символа в строку текста.

lab3_cpp

```
// lab3_main.cpp
// Лаб. №3. Вариант 1. Подсчёт вхождений заданного символа в строку текста.
// Алгоритм реализован в виде макроопределения COUNT_CHAR (см. lab3_asm.asm).

#include <stdio>
#include <windows.h>

extern "C" int CountCharAsm(const char* text, int length, char target); // обёртка над
макросом COUNT_CHAR

const int MAX_LEN = 100; // максимальная длина текста – константа, не зависящая от
фактического ввода
const int THRESHOLD = 5;

int main()
{
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    char buffer[MAX_LEN + 1];
    int length = 0;
    bool tooLong = false;

    printf("Введите текст (не более %d символов, признак конца ввода - точка):\n", MAX_LEN);

    int ch;
    while ((ch = getchar()) != EOF && ch != '.')
    {
        if (length < MAX_LEN)
        {
            buffer[length] = (char)ch;
            ++length;
        }
        else
        {
            tooLong = true;
        }
    }

    // Сразу после точки в потоке ввода остаётся "хвостовой" \n (от Enter, которым точка
    была отправлена).

    // Съедаем его здесь же, иначе следующий getchar() ниже прочитает именно его, а не
    нужный символ.
    while ((ch = getchar()) != '\n' && ch != EOF) {}

    buffer[length] = '\0';

    if (length == 0 || tooLong)
    {
```

```

    printf("\nВведённая последовательность символов не является текстом ");
    printf("(текст должен быть непустым и не длиннее %d символов).\n", MAX_LEN);
    return 0;
}

printf("\nВведённый текст (%d симв.): %s\n", length, buffer);

printf("Введите символ, вхождения которого нужно подсчитать: ");
int targetInt = getchar();
while (getchar() != '\n' && !feof(stdin)) {} // съедаем остаток строки ввода

if (targetInt == EOF)
{
    printf("Символ не введён.\n");
    return 0;
}

char target = (char)targetInt;
int count = CountCharAsm(buffer, length, target);

// Выбор слова "всего"/"целых" в зависимости от того, превышает ли count порог THRESHOLD
const char* amountWord = (count > THRESHOLD) ? "всего" : "целых";

// Выбор окончания "раза"/"раз": "раза" только для count == 2, 3 или 4; во всех
остальных случаях – "раз"
const char* timesWord = (count >= 2 && count <= 4) ? "раза" : "раз";

printf("Символ '%c' встречается в тексте %s %d %s.\n", target, amountWord, count,
timesWord);

return 0;
}

```

lab3_asm

```

;lab3_asm.asm
; Лаб. №3 (ассемблер, макроопределения). Вариант 1.
; Подсчитать количество вхождений заданного символа в строку текста.
;
; extern "C" int CountCharAsm(const char* text, int length, char target);
;
; Платформа проекта – x86 (Win32), сборка через ml.exe.

.386
.MODEL FLAT, C
.CODE

; -----
; Макроопределение COUNT_CHAR
; Параметры:
;   strAddr    – адрес начала строки (регистр или память)
;   strLen     – длина строки (регистр или память)
;   targetChar – искомый символ (регистр с его кодом)
;   result     – регистр-приёмник результата (счётчик вхождений)
;
; result должен быть регистром, ОТЛИЧНЫМ от esi/ecx – их макрос
; использует как собственные рабочие регистры (esi – указатель по
; строке, ecx – счётчик цикла).
; -----
COUNT_CHAR MACRO strAddr, strLen, targetChar, result
    LOCAL scanLoop, skipChar, scanDone ; свои метки на каждое место использования макроса
    —
    ; без LOCAL повторный вызов макроса в одном файле
    ; дал бы ошибку повторного определения метки

    mov esi, strAddr ; esi -> начало строки
    mov ecx, strLen  ; ecx = сколько символов ещё нужно проверить

```

```

xor result, result          ; result = 0 – обнуляем счётчик перед подсчётом

scanLoop:
    cmp ecx, 0
    je scanDone             ; строка кончилась – выходим
    cmp byte ptr [esi], targetChar
    jne skipChar            ; символ не совпал с искомым – пропускаем
    inc result               ; совпал – увеличиваем счётчик
skipChar:
    inc esi                  ; переходим к следующему символу строки
    dec ecx
    jmp scanLoop

scanDone:
ENDM

; -----
; int CountCharAsm(const char* text, int length, char target)
; Процедура-обёртка: принимает параметры по cdecl и вызывает макрос
; COUNT_CHAR, который разворачивается прямо здесь, в теле процедуры.
; -----
PUBLIC CountCharAsm
CountCharAsm PROC
    push ebp
    mov  ebp, esp
    push esi                  ; esi – callee-saved по cdecl, сохраняем

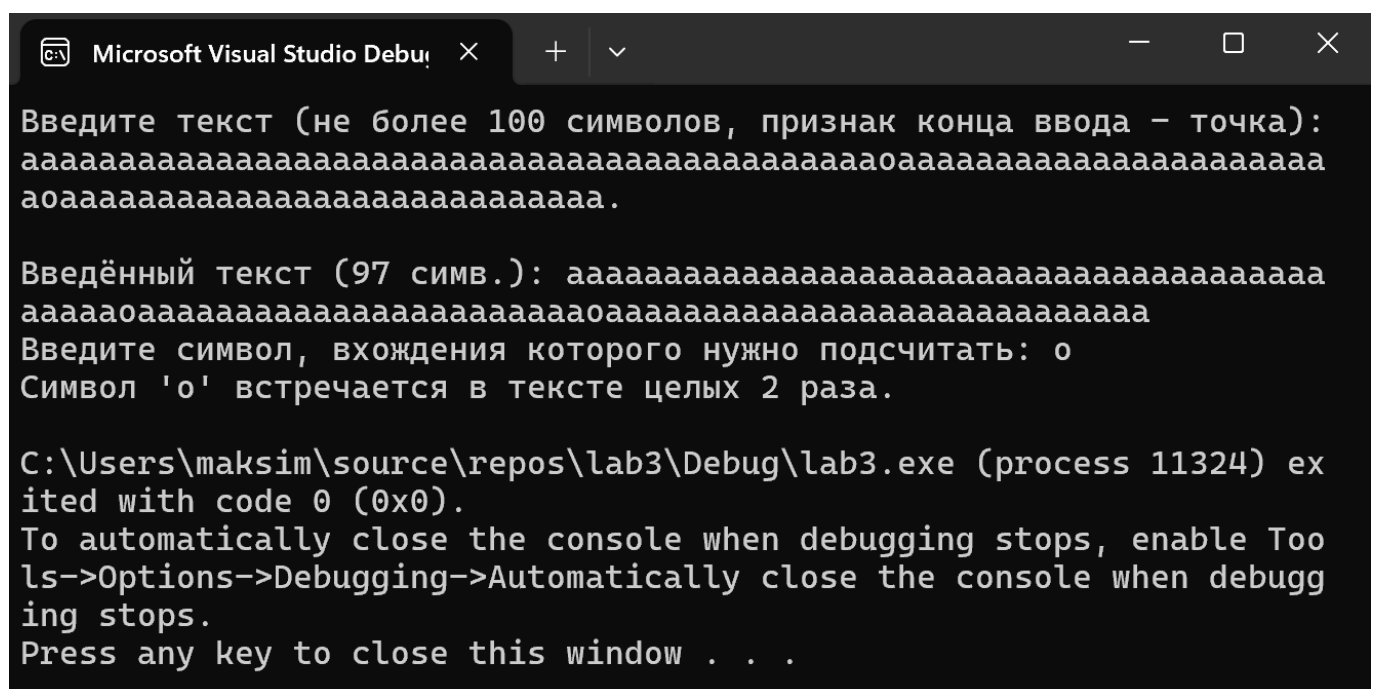
    mov dl, byte ptr [ebp+16] ; dl = target (char на стеке занимает минимум 4 байта, берём младший байт)

    COUNT_CHAR [ebp+8], [ebp+12], dl, eax ; strAddr=text, strLen=length, targetChar=dl, result=eax

    pop esi
    pop ebp
    ret
CountCharAsm ENDP

END

```



Microsoft Visual Studio Debug Console window showing the following output:

```

Введите текст (не более 100 символов, признак конца ввода – точка):
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aoooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo.

Введённый текст (97 симв.): aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaoooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooooo
Введите символ, вхождения которого нужно подсчитать: o
Символ 'o' встречается в тексте целых 2 раза.

C:\Users\maksim\source\repos\lab3\Debug\lab3.exe (process 11324) ex
ited with code 0 (0x0).
To automatically close the console when debugging stops, enable Too
ls->Options->Debugging->Automatically close the console when debugg
ing stops.
Press any key to close this window . . .

```

Вывод: В ходе выполнения лабораторной работы изучены приёмы разработки и использования макроопределений в ассемблере. Разработано макроопределение COUNT_CHAR, реализующее подсчёт количества вхождений заданного символа в строку текста (Вариант 1), с формальными параметрами: адрес строки, её длина, искомый символ и регистр-приёмник результата. Для предотвращения конфликта меток при возможном повторном использовании макроса в теле реализации применена директива LOCAL.

Макрос протестирован через процедуру-обёртку CountCharAsm, в теле которой происходит его развёртывание компилятором ассемблера непосредственно в месте вызова, без формирования отдельного адреса перехода (call/ret), что отличает механизм макроподстановки от обычных процедур, использованных в предыдущих лабораторных работах. Работоспособность макроопределения подтверждена серией тестов на различных входных строках и искомых символах, включая случаи нулевого, единичного и множественного количества вхождений — во всех случаях результат совпал с ожидаемым при ручном подсчёте.